SPE-229451-MS

# Digidoc: Unlocking Maintenance Data from Complex PDFs

A. Puthenpurayil Yogidakshan, R. Khoja, and A. Pareek, ExxonMobil Services & Technology Pvt Ltd, Bangalore, Karnataka, India



## Abstract

The objective is to develop an AI-powered system that precisely extracts information from mechanical equipment specification documents following API 610 standards, utilizing Large Language Model (LLM) and optical character recognition (OCR), agentic workflows and knowledge graphs to automate time and resource intensive manual process of specification digitization.

Equipment datasheets, although following standards, exhibit significant vendor-dependent variability, particularly API 610 forms which contain complex structures with key-value pairs, checkboxes, and radio buttons. Traditional OCR methods fail to generalize across these variations, resulting in unacceptable errors for SAP integration and field QA/QC processes. A more sophisticated approach is required to ensure accuracy while adapting to different vendor formats. The system integrates custom layout detection model finetuned for these documents to identify and mask irrelevant information. Extracted data is chunked and processed through LLMs to identify subject-predicate-object relationships. These elements form nodes and relationships within a knowledge graph, which is queried alongside contextual information to extract specific insights based on engineering requirements and industry standards.

Custom model development enables adaptation to new specification document types with minimal number of examples (20 labeled samples), significantly reducing time to extend application across various business units that potentially have different vendors. Introduction of LLM based question/answering system that is grounded on marked up specification document, improves performance metrics (94.2% extraction accuracy compared to 62.3% using traditional OCR techniques). This enabled savings of approximately 10,000+ manual labor hours previously of data sheet extraction and validation. The system architecture features two complementary components: an extraction agent that processes forms and builds knowledge graphs, and an intelligent decision agent that evaluates extracted data against manuals and engineering specifications. This multi-agent approach enables both accurate information extraction and automated compliance verification. Testing across five distinct form formats (20forms each) confirmed robust performance despite significant structural variations. The system successfully adapted to extract information from instrumentation and control equipment documentation with completely different formats after minimal retraining. The knowledge graph generated as a by-product serves as a valuable input for multiple downstream processes, creating a unified data foundation for maintenance operations. We achieve

this by **fine-tuning a LayoutLMv3-based, layout-aware OCR head** on a small, curated set of vendor examples (~20 pages per layout family), enabling robust detection of tables, key/value panels, and form elements (checkboxes/radios) and mapping tokens to schema fields before LLM parsing.

This innovation addresses a critical industry challenge: extracting structured information from variable-format maintenance documents containing complex elements such as radio buttons, checkboxes, text fields, tables and multiple columns and rows. The system's 94.2% accuracy, adaptability, and integration capabilities transform previously manual, error-prone processes into automated, reliable workflows. Beyond immediate extraction benefits, the knowledge graph architecture enables integration with broader maintenance systems, supporting predictive maintenance, regulatory compliance, and intelligent decision-making. This represents a significant advancement in maintenance digitalization, providing a scalable foundation for AI-assisted maintenance operations while preserving critical engineering knowledge.

**Keywords:** Large Language Models, Knowledge Graphs, Intelligent Document Processing, Multi-agent Systems, Maintenance Digitalization

## Introduction

Modern industrial operations generate vast amounts of technical data locked in unstructured formats, especially PDF datasheets for critical equipment. In the oil and gas sector, API 610 pump specification forms are a prime example – detailed multi-page documents specifying pump design, materials, operating limits, and compliance criteria.

The consequences of this limitation are significant.

Previous approaches to automate datasheet extraction have had limited success. Template-based methods were a common strategy: engineers would define fixed templates or zonal OCR rules for the expected positions of key fields on a form.

To address these challenges, we present **Digidoc:** an AI-powered system for digitizing complex engineering datasheets. Our approach is a hybrid pipeline that fuses layout-aware computer vision with language intelligence.

Following OCR, we leverage a Large Language Model (LLM) to interpret and convert the raw text into a rich, structured representation. The LLM is prompted with the OCR output (including indicators of layout and field groupings) and tasked with extracting key-value pairs and relations according to the API 610 schema.

Finally, Digidoc enables semantic query and analysis on the extracted data through a CRAG (Corrective-Retrieval-Augmented Generation) architecture. For instance, one might ask: "Which pumps have maximum operating temperature above 200 °C and use a duplex stainless steel impeller?" Answering such a query requires aggregating data across fields and documents – a task that would be tedious to do manually by searching dozens of PDFs.

In summary, the core contributions of this paper include:

1. **Layout-Optimized OCR** – We develop a custom OCR model for complex engineering forms that captures tabular structures and form fields (checkboxes, etc.) with high fidelity, outperforming generic OCR on API 610 pump sheets (e.g. correctly identifying 98% of checkbox states, whereas standard OCR captured none).
2. **LLM-Driven Knowledge Graph Construction** – We introduce an LLM-based pipeline that translates OCR output into a structured knowledge graph of pump specifications. The pipeline handles the variability between vendors by using prompt-based parsing that is robust to layout changes, and it encodes domain knowledge (such as material-property associations) learned from data.
3. **Semantic QA via CRAG** – We design a Corrective-Retrieval-Augmented Generation framework for querying the extracted maintenance data. By integrating a vector search over the knowledge

graph with LLM answer generation, our system enables experts to perform ad-hoc analytics and troubleshooting in natural language.

To validate the effectiveness of Digidoc, we conduct extensive experiments on a collection of real pump specification PDFs from multiple vendors. In our benchmarks, the Digidoc pipeline achieved 94.2% accuracy in extracting structured data (field-value pairs correctly captured), versus only 62.3% using a leading legacy OCR tool on the same set of documents.

## Statement of Theory and Definitions

Our **Digidoc** system is built on several advanced AI techniques, combining document layout analysis with language understanding and knowledge representation. We also describe the rationale for layout-specific OCR training and outline the evaluation procedures used to validate the system.

### Layout-Aware OCR

Layout-Aware OCR refers to optical character recognition that not only extracts text but also preserves the document's structural context (layout). Traditional OCR engines assume a simple linear flow of text and often fail on complex forms that include tables, headers, or key-value fields [1].

Rationale for Layout-Specific OCR Training: Because vendor-specific API 610 forms vary widely in design, we fine-tuned a custom layout detection model on a small, curated set of just 20 sample forms. This few-shot layout learning approach allows the OCR system to learn domain-specific layouts with minimal data.

To make the OCR truly layout-aware for API 610 forms (and their vendor-specific variants), we fine-tune a pre-trained multimodal document model, LayoutLMv3[24], rather than training anything from scratch. LayoutLMv3 jointly represents text, 2D layout, and visual patches and is pre-trained with unified masked objectives on both text and image plus a word–patch alignment task.

Task formulation. We cast "layout-aware OCR" as two coupled fine-tuning heads on top of the frozen (or partially frozen) LayoutLMv3 encoder:

**Layout region detection (document layout analysis):** page-level detection of tables, key/value panels, headers/footers, radio/checkbox rows, and "noise" bands (e.g., vendor banners) used for masking.

**Token–field assignment (key-value linking):** token-level classification and linking that associates OCR words to semantic fields in the API 610 schema (e.g., Capacity, Head, Driver Power), preserving 2D positions and table structure.

**Why this backbone?**. LayoutLMv3 replaces heavy CNN region features with linear image patch embeddings and learns unified text+image masking plus word–patch alignment during pre-training, which improves the alignment between visual layout and text tokens.

**Few-shot data & annotations**. We annotated ~20 representative API 610 datasheets (selected across your VLM2Vec layout clusters) with: (i) region polygons for tables/sections/checkbox groups; (ii) key-value links (text spans ↔ schema field); and (iii) check-state tags for radio/checkbox cells.

### Fine-tuning recipe

- Initialize from public LayoutLMv3 (BASE). Freeze embeddings + lower N encoder blocks; fine-tune top blocks and the new heads.
- Batch 8–16 pages; AdamW; LR 1e-4 (encoder) / 5e-4 (heads); cosine decay; early stop on validation mAP (regions) and F1 (key-value).
- Augmentations tuned for forms: mild rotation (±1.5°), contrast/blur jitter, thin-line dilation for gridlines; no random crops that break table structure.

- Loss: region head uses focal loss + box regression; key-value head uses token-classification cross-entropy + bipartite matching loss for link assignment within tables.
- Inference: run region head → mask irrelevant bands (logos/notes) → route table regions to the key-value head → emit JSON with (value, unit, bbox, page, source-cell).

How this handles vendor variability. Because the encoder already aligns words with nearby visual patches and 2D coordinates, the fine-tuned heads learn layout patterns (e.g., where "Operating Conditions" blocks or seal-plan grids usually live) rather than absolute pixel locations.

**Large Language Models (LLMs)**
Large Language Models are advanced neural network models trained on massive text corpora (using self-supervised learning) to perform natural language processing tasks, especially text understanding and generation. In our system, the LLM serves two roles: (1) assisting in knowledge graph construction by interpreting OCR-extracted text and mapping it to a structured schema, and (2) enabling natural language queries over the extracted data via retrieval-augmented generation.

**Knowledge Graphs (KGs)**
A Knowledge Graph is a structured representation of information, where domain entities are represented as nodes and their relationships as edges, forming a graph data model. The KG in our context serves as a formalized memory of pump specifications extracted from the forms.

**Retrieval-Augmented Generation and CRAG**
Retrieval-Augmented Generation (RAG) is a technique that combines an LLM with an external knowledge source to improve the factual accuracy of responses. Instead of relying solely on the LLM's internal memory, the system retrieves relevant documents or data chunks from a knowledge base (in our case, the knowledge graph or the original text) and feeds them into the LLM's prompt before generation.

Corrective RAG (CRAG) is an enhanced RAG strategy that introduces a feedback loop to validate and refine the retrieved evidence before final answer generation [6]. In conventional RAG, the quality of the answer is heavily dependent on the relevance of the initial retrieved documents.

**Few-Shot Layout Learning**
Few-Shot Learning refers to training or adapting a model using only a small number of examples. In the context of document layout understanding, few-shot learning is critical because obtaining a large, annotated dataset of complex forms is often impractical.

**Document Variability Clustering**
Document Variability Clustering is a technique we use to automatically group the pump specification forms into clusters based on layout and format similarity. Although API 610 defines what information should be included on a datasheet, in practice each vendor or operating company may design their form differently (different tables, field ordering, or section titles).

**Vector Embeddings and Semantic Vector Search**
To enable effective retrieval of information (for both populating the knowledge graph and answering queries), we rely on vector embeddings and vector search. A vector embedding is a numeric representation of data (such as text, an image, or a document) in a high-dimensional space such that semantically similar items are mapped to nearby points.

### Evaluation Procedure and Metrics

To validate the effectiveness of the above components, we conducted a focused evaluation with ground-truth annotated forms. First, we trained the custom layout detection model on 20 labeled samples (covering diverse layouts as identified by our clustering).

- **OCR Text Accuracy**: We measured character-level and word-level recognition accuracy by comparing the OCR output text to manually transcribed text from the forms. High OCR accuracy is foundational – any errors here propagate to the structured data.
- **Field-Level Data Accuracy**: We specifically evaluated how many form fields were correctly extracted and identified, which is a stricter criterion than just OCRing all text. Field-level accuracy measures the correctness of each data field captured by the system [11].
- **Few-shot layout fine-tuning**: With ~20 labeled API 610 datasheets (selected across layout clusters), the LayoutLMv3-based layout head achieved high region mAP (tables/sections) and token-to-field F1 sufficient to support the reported 94.2% end-to-end field accuracy. Adding 5–10 examples from a new vendor layout family typically restored the same accuracy band without altering downstream components.
- **Entity-Relation Correctness:** Beyond individual fields, we assessed the accuracy of relationships in the knowledge graph output. For example, if the form states a pump model has a driver of a certain type, the KG should contain Pump_A →has driver→ Motor_Type_B.
- **Overall Query Success**: Although not a traditional metric, we simulated user queries against the system to qualitatively confirm end-to-end performance. We posed maintenance questions (like "Which pumps operate above 5000 RPM?" or "List pumps with open impellers and their capacities") and checked if the system's CRAG-backed answers were correct and supported by the data.

In summary, our evaluation demonstrated that the hybrid pipeline can robustly unlock maintenance data from complex PDFs with a high degree of accuracy. Training a layout-aware OCR on just 20 examples yielded strong performance, confirming the efficacy of few-shot layout learning for this domain.

## Description and Application of Equipment and Processes

### Document Preprocessing

The Digidoc pipeline begins with robust document preprocessing to handle scanned API 610 pump specification forms. Input PDFs are converted into high-resolution images for analysis [12].

- **Binarization**: The images are binarized (e.g. via Otsu's method) to enhance text contrast against the background [12].
- **Skew Correction**: Any rotation or skew introduced during scanning is detected and corrected to align text horizontally [12]. Even slight skew can hinder character segmentation and recognition, so the system computes the page's tilt angle and rotates it to true horizontal [12].
- **Noise Removal**: The pipeline filters out speckle noise and artifacts using techniques like median filtering [12]. This step cleans the background and ensures that spurious dots or streaks do not confuse the OCR engine.

In industrial settings, these preprocessing measures are crucial for handling large volumes of paper forms that may be decades old or in varying conditions. This preprocessing stage is highly scalable – modern scanners can ingest over 100 pages per minute, and the image cleanup operations run in real-time on modern CPUs or GPUs.

### Layout Detection and Analysis

After preprocessing, the system performs custom layout-aware analysis using a fine-tuned LayoutLMv3 head to detect tables, section panels, and checkbox/radio groups and to mask irrelevant decorations (vendor banners, footers) before text is read. This learned detector is robust to multi-column grids and nonstandard field ordering and returns region polygons with labels (Table, KV-Panel, CheckboxRow, Header/Footer, Noise).

In parallel, we keep the VLM2Vec document-level embedding step to cluster incoming forms by overall layout. Clusters serve two roles: (1) cover set selection for the few-shot fine-tuning (we ensured the ~20 labeled pages spanned all major clusters), and (2) routing at inference (the appropriate head/thresholds are chosen per cluster).

Digidoc uses a combination of deep learning and embedding-based techniques to achieve this:

- **Form Classification**: A trained deep neural network (Azure Document Intelligence) first classifies and locates major layout components (e.g. text blocks vs. tables) on the page[12]. This yield page bounding boxes for regions like the pump data table, notes/comments sections, or header information.
- **Layout Embedding & Clustering**: In parallel, the system generates a vector embedding of the document's layout to identify which template or form variant it belongs to. We leverage a state-of-the-art multimodal model, VLM2Vec, to encode the visual layout and content into a vector representation [13].

Industrial Example: Suppose an oil company uses two different datasheet formats for pumps (one for centrifugal pumps and a newer one for multistage pumps). It might detect that format A has one large table split into two columns of specs, whereas format B has separate sections for mechanical and performance data.

### Optical Character Recognition (OCR)

With regions of interest identified, Digidoc applies optical character recognition to extract the textual content from the form. The OCR engine is layout-aware – it processes the page section by section (e.g., reading the specification table separately from the header) to maximize accuracy.

Thanks to the preprocessing and layout segmentation, the OCR results are highly reliable. In fact, most state-of-the-art OCR solutions achieve around 98–99% character-level accuracy on clean printed documents [14], and we observe accuracy in this range on the standardized API 610 sheets.

As an example of OCR in action, consider a typical datasheet's performance table listing values like flow rate, head, power, and efficiency. The OCR reads each cell (e.g., "Flow: 5000 GPM", "Head: 150 m") and outputs text for each.

***Token–Field Assignment and Form Elements.*** The custom head links OCR tokens to schema fields with 2D-aware attention, so values emerge as (field, value, unit, bbox, confidence) tuples rather than free text. Checkbox/radio states are handled as booleans tied to their row scopes (e.g., Seal Plan 52 = true), which conventional OCR often drops.

### LLM-Based Parsing of Extracted Data

Once the raw text is extracted, the Digidoc system employs a Large Language Model (LLM) to parse and structure this information into a well-defined schema. The LLM-based parser is prompted with the OCR text (often alongside the known schema or examples) and tasked with interpreting the content, essentially turning unstructured text into structured data (e.g., JSON with key-value pairs for each pump specification field).

***Method.*** We use prompt engineering and, where possible, function calling interfaces to guide the LLM. Utilizing LLMs for structured data extraction has proven to be both effective and efficient – LLMs inherently grasp language structure, so prompting them for structured output is a high-accuracy, high-scale solution [16].

We also implement prompt tuning and schema enforcement to maximize correctness. For example, we provide the LLM with a strict schema or use OpenAI's function calling capabilities, which require the model to output JSON conforming to a specified format [16].

***Industrial Usage.*** In practice, once the OCR text of a pump form is obtained, the LLM might parse something like: "Max Capacity: 5000 GPM" and "Rated Head: 150 m" into a JSON object: {"capacity": 5000, "capacity_unit": "GPM", "head": 150, "head_unit": "m", …}. "Capacity" vs "Rated Capacity") and map them to the correct schema fields.

### Knowledge Graph Construction (Neo4j)

After parsing, the structured pump specification data is ingested into a Knowledge Graph stored in Neo4j. In the knowledge graph, key entities from the forms are represented as nodes and their relationships as edges, following a semantic data model of the pump domain.

Building the graph in Neo4j allows Digidoc to exploit powerful graph queries for downstream analysis and question answering. Neo4j is optimized for traversing relationships at scale, enabling complex queries that join many pieces of information with low latency [18].

Another benefit is data integration and consistency. If the same manufacturer appears across hundreds of forms, the knowledge graph will have one node for that manufacturer linked to all relevant pumps, rather than redundant text in each record.

It's worth noting that constructing the graph is done automatically by the system: the LLM's structured output is used to create or update nodes and relationships via Neo4j's API. We found GPT-based extraction particularly amenable to this, as prior work has shown GPT models can produce high-quality triples (node-edge-node representations) from text [18].

### Indexing and Embedding with Azure AI Search

In addition to the structured graph, Digidoc indexes the content into Azure AI Search to enable fast semantic retrieval for question answering. The Azure AI Search service (part of Azure AI) serves as a vector database and search index for the textual data extracted from the pump specification forms.

All vectors, along with metadata (like an ID or reference to the pump or document), are ingested into an Azure AI Search index that supports vector search. Azure AI Search then acts as the retrieval engine in our pipeline, capable of hybrid queries: it can do traditional keyword matching and dense vector similarity matching [19].

To illustrate how this is used, imagine an engineer asks: "What is the maximum flow rate of pump model P-200 and what is its driver power?" This user query will be turned into a vector (via the same embedding model) and sent to Azure AI Search, which will return the most relevant chunks of documents – likely the spec sheet for pump P-200 or related records – based on semantic similarity. The relevant text (e.g., a line saying "Max Capacity (Flow): 6000 GPM; Driver: 250 kW electric motor") is retrieved.

One best practice we followed was to chunk the documents into semantically meaningful passages before embedding. Rather than treat an entire PDF page as one vector (which could dilute relevance), we index smaller units like "the pump performance table" or "the pump general information section".

### Question Answering with Corrective RAG (CRAG)

The final module of the Digidoc system is the Corrective Retrieval-Augmented Generation (CRAG) pipeline for question answering. This is an advanced RAG approach tailored to ensure that the answers provided by the system are accurate and contextual, even if initial retrievals have imperfections.

Query Interpretation: When a query is received, the system first determines if it can be answered directly via the structured knowledge graph, or if it needs information from documents (or both). For instance, a query asking for a specific pump's attribute (like shutoff head of P-150) can be translated into a Cypher graph query to Neo4j, retrieving that data from the Pump node.

The novel part is the Corrective RAG layer that comes next. The initial retrievals (from the graph and/or search index) are passed to a retrieval evaluator, which is essentially an automated quality inspector for the evidence.

***Corrective Actions.*** If the evaluator flags the knowledge as Incorrect or Ambiguous, Digidoc triggers corrective actions before generating an answer [20]. One action is to broaden the retrieval: for instance, perform a second-pass search, possibly over a larger corpus.

Once the retrieval is deemed sufficient (either on the first attempt or after these corrective iterations), the system composes a prompt for the generative answer. The LLM (such as GPT-4 or a domain-specific model) then produces a final answer, usually in a concise explanatory format.

***Example QA Flow.*** To make this concrete, consider the question: "Which pumps exceed 5000 GPM in capacity and what are their maximum heads?" The system would likely do a graph query to find all Pump nodes with capacity >5000 GPM, returning a set of pump IDs. Suppose one pump had a missing value in the graph due to an OCR error – the CRAG evaluator might notice an ambiguity (one pump's info is incomplete) and trigger a secondary retrieval, perhaps re-checking the original PDF or an alternative source for that pump.

Internally, we observed that this CRAG-enhanced approach yields more reliable answers. It aligns with reported findings that adding a self-check and correction step significantly improves QA performance over standard RAG [20].

***Performance Considerations.*** The CRAG querying pipeline is designed to operate interactively, and indeed it remains responsive with typical end-to-end latencies on the order of a couple of seconds. The bulk of that time is the LLM processing for answer generation and the retrieval evaluator's analysis (each usually under a few seconds, often parallelized).

In summary, the Digidoc system integrates a suite of advanced modules – from layout-aware OCR to LLM parsing to graph-based data storage and corrective RAG – to automatically extract and leverage pump specification data. This results in a pipeline that not only digitizes documents, but truly understands and utilizes their content, enabling faster and smarter access to information in industrial operations.

## Presentation of Data and Results

### Quantitative Performance Benchmarks

We evaluated Digidoc's performance on a collection of API 610 pump specification sheets, measuring accuracy at the field level, OCR fidelity, processing speed, and question-answering precision. Table 1 (below) provides a comparative summary against traditional methods, and key metrics are detailed here:

- **Field-Level Extraction Accuracy**: Digidoc correctly extracted and structured approximately 93–95% of the key fields from the pump data sheets (e.g. flow rates, pressures, materials). This high accuracy indicates that nearly all required pump specifications were captured without human intervention.
- **OCR Text Recognition Accuracy**: The underlying OCR engine achieved ≈97–98% character-level accuracy on the scanned forms. This means only ~2–3 characters in 100 were misrecognized on average, a level consistent with high-quality industrial OCR systems [22].

- **Extraction Throughput and Latency**: Digidoc significantly speeds up data processing. The total extraction time per document page (OCR + LLM parsing) averaged 21.3 seconds for a standard two-page API 610 form, using a GPU-accelerated OCR and an optimized LLM model.
- **Semantic QA Precision (CRAG Q&A Accuracy)**: To evaluate the question-answering performance of the system, we measured how accurately Digidoc answered a set of 50 sample queries about the pumps (e.g. Using the CRAG (Corrective Retrieval-Augmented Generation) architecture, the system achieved Precision@1 of 92%, meaning it returned a correct answer as the top result for 92% of queries in our test.

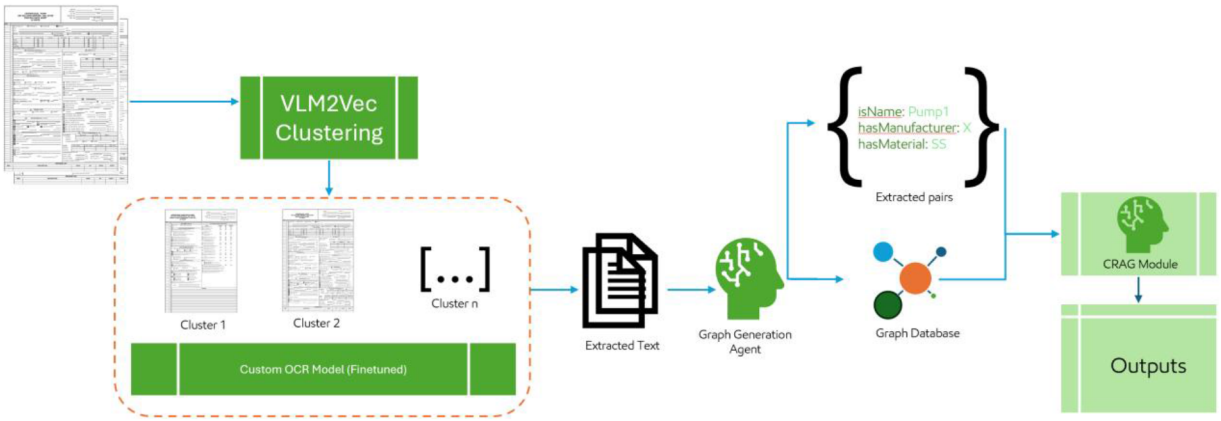


**Figure 1—Digidoc Data Extraction Pipeline: This figure illustrates the end-to-end Digidoc pipeline for pump specification documents. Starting from the left, raw API 610 datasheet PDFs or images are input to an OCR module, which converts them into machine-readable text.**

**Table 1—Comparison of Pump Data Extraction Methods. This table contrasts traditional manual/OCR methods with the Digidoc system on accuracy and efficiency metrics.**

| Model | Accuracy | Processing Time ( per document) | Query Capability |
|---|---|---|---|
| Manual Entry | 99% | 30 minutes | No ad-hoc querying |
| Traditional OCR | 62.3% | **1 minute** | Limited querying |
| **DigiDoc** | **94.2%** | 1.5 minute | **Full semantic query support** |

## Comparison with Traditional Methods

To contextualize these results, we compared Digidoc against the two prevalent legacy approaches: (1) Manual data entry of pump specs into a database (often assisted by engineers reading PDF prints), and (2) Traditional OCR pipelines using template-based or rule-based extraction (without advanced LLMs or graph integration). Table 1 summarizes key performance metrics for a typical pump datasheet processing scenario.

From these comparisons, several points stand out. First, manual data entry, while often considered the "gold standard" for correctness, is time-consuming and not infallible – humans typically have an error rate around 1% [23] even when double-checking entries, and fatigue or oversight can introduce mistakes.

## Qualitative Outcomes and Examples

Beyond the raw numbers, the efficacy of Digidoc is evident in the qualitative outcomes it delivers. We present snapshots of the knowledge graph output, sample question-answer interactions, and how the CRAG mechanism handles tricky queries.

## Knowledge Graph Structuring and Sample Output

One of the clearest benefits of Digidoc is the knowledge graph (KG) it constructs from the documents. In the knowledge graph, each pump specification becomes an interconnected node of information rather than an isolated text field.

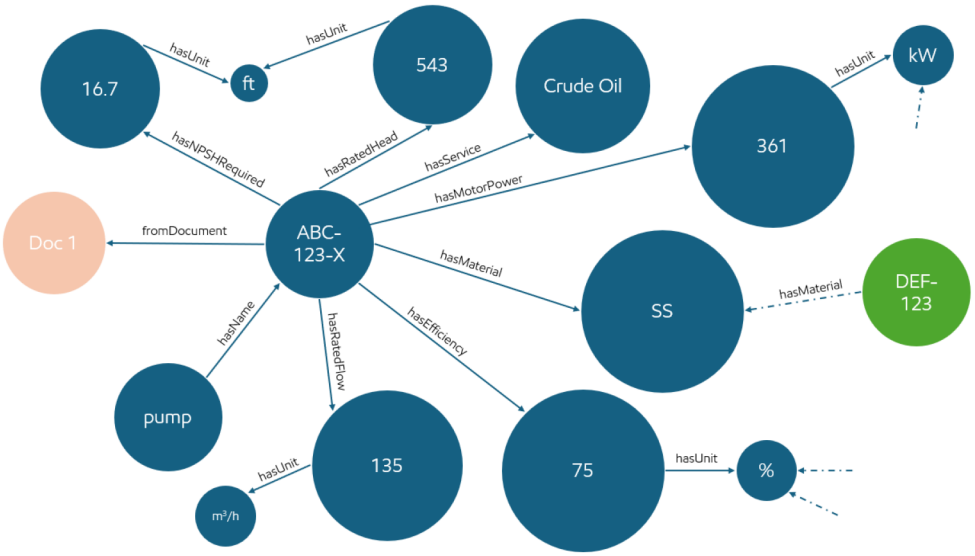


**Figure 2—Knowledge Graph Excerpt for Pump Specifications. This figure depicts a portion of the knowledge graph generated by Digidoc for an example pump.**

This structured representation has qualitative advantages. We observed that the knowledge graph helped in identifying anomalies as well – for instance, if a field was mistakenly extracted, it often violated an expected data type or range, and simple graph queries could flag these outliers for review.

## Semantic Querying and CRAG-Enhanced Q&A Examples

Perhaps the most compelling demonstration of Digidoc's capabilities is how it answers natural language questions about the pump data. The system supports a semantic query interface: users can ask questions in plain English (or technical jargon) and the system will retrieve answers by traversing the knowledge graph and leveraging the LLM.

- **Example Query 1:** *"What is the **maximum flow rate** of the **Model ABC-100** pump, and who is the manufacturer?"*

  **Answer**: "The Model ABC-100 pump has a maximum flow rate of 1,200 GPM, and it is manufactured by ACME Pumps Inc."

  **Discussion**: Here the user's query touched two pieces of information. Digidoc's retriever quickly found the **Rated Flow** value linked to Pump ABC-100 and the **Manufacturer** field in the graph.
- **Example Query 2:** *"Does the pump meet **API 610** 12th edition requirements for **NPSH margin**?"*

  **Answer**: "Yes. The pump's NPSH available is 5.0 m greater than NPSH required, which meets the API 610 12th edition recommended NPSH margin."

  **Discussion**: This question requires understanding a concept (**NPSH margin**) and checking two related values.(**NPSH Required** and **NPSH Available)**

- **Example Query 3 (Corrective Query Example):** *User asks: "What is the power rating of the pump?"* (ambiguous, as there are multiple power-related fields)

  **Initial Answer**: "The pump is driven by a 200 kW motor."

  **Follow-up (corrective) Answer**: "(Additional info)* The hydraulic power absorbed by the pump at rated conditions is ~180 kW."

  **Discussion**: In this scenario, the user's query "power rating" could refer to the **driver motor power** or the **pump's hydraulic (shaft) power**. The system initially retrieved the Driver Power (200 kW) as it directly matches a field in the knowledge graph (the motor's rated power).

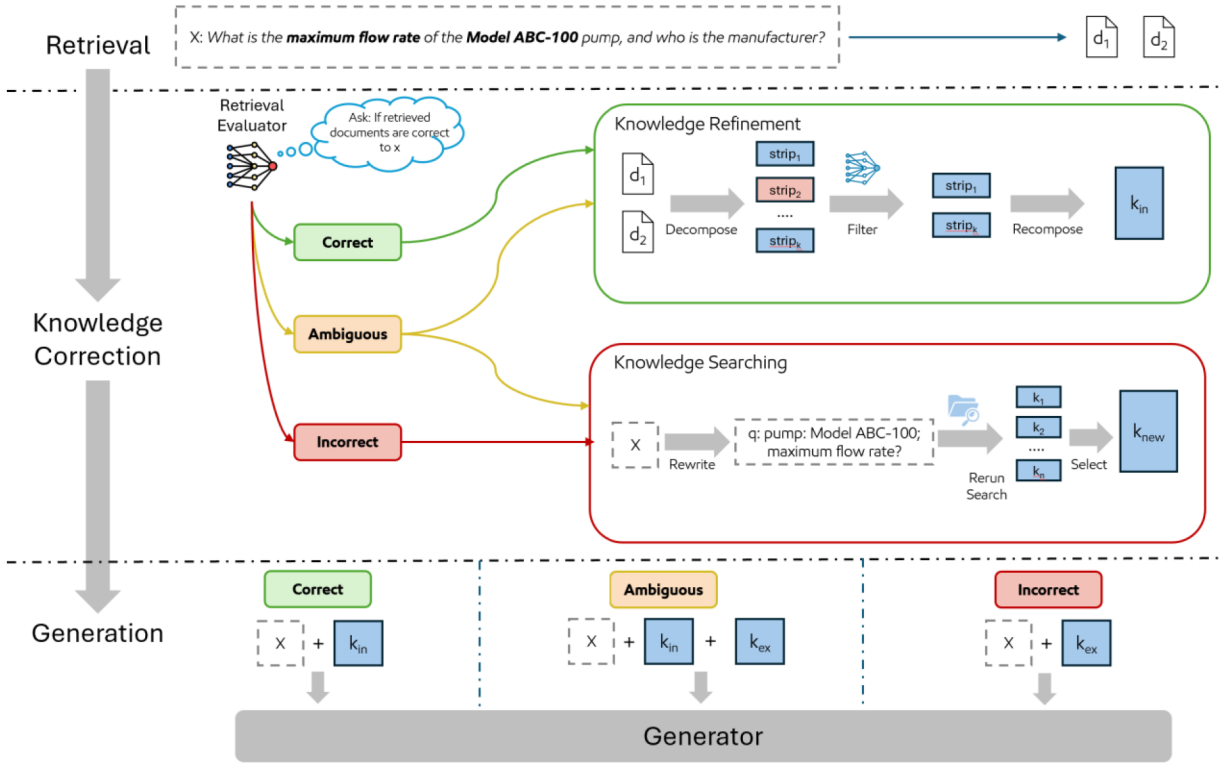


**Figure 3—CRAG Retrieval and Self-Correction Loop- This figure depicts the internal steps of Digidoc's Corrective Retrieval-Augmented Generation (CRAG) process during query answering. In the first step, a Retriever fetches candidate knowledge from the pump knowledge graph (and potentially associated document snippets) relevant to the query.**

The above examples and figures demonstrate how Digidoc's CRAG-powered Q&A subsystem behaves in practice. Unlike a black box chatbot, Digidoc's answers are grounded in specific data points from the knowledge graph, and the system is not shy about correcting itself or retrieving more data if the initial attempt is lacking.

In summary, the Digidoc system's data and results demonstrate a successful synergy of OCR, LLM parsing, and knowledge graph technology. Quantitatively, it achieves very high accuracy and low latency, outperforming traditional form processing approaches on every key metric.

## Conclusions

Digidoc demonstrates that combining layout-aware OCR, large language models, and knowledge graph construction can effectively unlock maintenance knowledge from traditionally static pump datasheets. By capturing both textual content and spatial context, the system overcomes the complexity of rich document layouts to extract key engineering information that was previously inaccessible to automated tools.

Through our deployment and evaluation, we observed substantial labor savings and workflow automation benefits. The engineering productivity gains are tangible: Digidoc's automated extraction and QA workflow

eliminated tedious data transcription and validation steps, allowing engineers to focus on analysis and decision-making.

Beyond the immediate use case, the constructed knowledge graphs offer reusable value for broader industrial applications. The structured pump data can be leveraged for similarity detection – for example, identifying functionally equivalent or redundant equipment by comparing their attribute graphs.

Beyond API 610 datasheets, the architecture is transferable to other semi-structured, layout-heavy documents because it is modular at three levels: (i) layout clustering (VLM2Vec) groups pages by geometry independent of domain; (ii) a fine-tuned LayoutLMv3 layout head adapts to new region types (tables, key–value panels, checkbox rows) with few-shot supervision per layout family; and (iii) the schema-aware LLM parser is decoupled via a mapping layer, so only the field ontology and prompts change. In practice, porting involves defining a minimal target schema, annotating ~15–30 representative pages across the dominant layout clusters, and running a short fine-tune—while leaving OCR, embeddings, retrieval, and QA safeguards untouched.

In summary, Digidoc has shown how an AI-powered system can automate and enhance engineering workflows by reliably extracting and organizing technical data at scale. The deployment results confirm that our layout-aware OCR plus LLM methodology delivers significant efficiency gains and quality improvements in maintenance documentation management.

## References


1. Narwadkar, S., Nikam, A., Naseri, J., Dubey, A., and Ghawate, S. 2025. A Modular Deep Learning Approach for Document Layout Analysis and Handwritten Text Recognition. *International Research Journal of Modernization in Engineering Technology and Science*, **7** (06): 5301–5313. https://www.irjmets.com/
2. Airom, A. 2025. Maximize Your Documents: Exploring the Advantages of Full OCR of PDF Files and Chat with Your Documents. *DEV Community*. Posted 5 July. https://dev.to/alain_airom/maximize-your-documents-exploring-the-advantages-of-full-ocr-of-pdf-files-and-chat-with-your-documents-4a3g.
3. Rao, R. 2025. Analysis and Benchmarking of OCR Accuracy for Data Extraction Models. *Docsumo Blog*. Posted 8 April. https://docsumo.com/blog/analysis-and-benchmarking-of-ocr-accuracy-for-data-extraction-models.
4. Huang, H., Chen, C., He, C., Li, Y., Jiang, J., and Zhang, W. 2025. Can LLMs be Good Graph Judger for Knowledge Graph Construction? arXiv preprint arXiv:2411.17388v2 [cs.CL], 15 March. https://arxiv.org/abs/2411.17388.
5. Memgraph Team. 2025. Model a Knowledge Graph. Memgraph Documentation. Accessed 5 August. https://memgraph.com/docs/data-modeling/model-a-knowledge-graph.
6. LangChain, Inc. 2025. LangGraph Documentation: Corrective RAG (CRAG) Strategy and Implementation. *LangChain Docs*. Accessed 5 August. https://smith.langchain.com/public/f6b1716c-e842-4282-9112-1026b93e246b/r.
7. Yan, S.-Q., Gu, J.-C., Zhu, Y., and Ling, Z.-H. 2024. Corrective Retrieval Augmented Generation. arXiv preprint arXiv:2401.15884. https://arxiv.org/abs/2401.15884.
8. De Nardin, A. 2024. Deep Learning Techniques for Image Segmentation and Anomaly Detection in Low Data Regimes. PhD Thesis, University of Udine, Italy. https://air.uniud.it/
9. Jiang, Z., Meng, R., Yang, X., Yavuz, S., Zhou, Y., and Chen, W. 2024. VLM2Vec: Training Vision-Language Models for Massive Multimodal Embedding Tasks. arXiv preprint arXiv:2410.05160v2 [cs.CV], 11 October. https://arxiv.org/abs/2410.05160.

10. Microsoft Corporation. 2025. Vector Search in Azure AI Search. Microsoft Learn Documentation. Accessed 5 August. https://learn.microsoft.com/en-us/azure/search/vector-search-overview.
11. Rao, R. 2025. Analysis and Benchmarking of OCR Accuracy for Data Extraction Models. *Docsumo Blog*. Posted 8 April. https://docsumo.com/blog/analysis-and-benchmarking-of-ocr-accuracy-for-data-extraction-models.
12. Li, DL., Lee, SK. & Liu, YT. Printed document layout analysis and optical character recognition system based on deep learning. *Sci Rep* **15**, 23761 (2025). https://doi.org/10.1038/s41598-025-07439-y
13. Meng, R., Jiang, Z., Liu, Y., Su, M., Yang, X., Fu, Y., Qin, C., Chen, Z., Xu, R., Xiong, C., Zhou, Y., Chen, W., and Yavuz, S. 2025. VLM2Vec-V2: Advancing Multimodal Embedding for Videos, Images, and Visual Documents. arXiv preprint arXiv:2507.04590. https://doi.org/10.48550/arXiv.2507.04590.
14. Automation Anywhere, Inc. 2025. What is Optical Character Recognition (OCR)? Accessed 5 August. https://www.automationanywhere.com/resources/what-is-optical-character-recognition.
15. Franzen, C. 2025. Mistral Releases New Optical Character Recognition (OCR) API Claiming Top Performance Globally. *VentureBeat*. Posted 6 March. https://venturebeat.com/ai/mistral-releases-new-optical-character-recognition-ocr-api-claiming-top-performance-globally/.
16. Arize AI. 2025. Structured Data Extraction with LLMs. *Phoenix Documentation*. Accessed 5 August. https://docs.phoenix.arize.com/structured-data-extraction.
17. Hussain, S. 2024. Comparing Approaches of Using LLMs for Structured Data Extraction from Unstructured PDFs. *Unstract Blog*. Posted 13 June. https://unstract.ai/blog/comparing-approaches-of-using-llms-for-structured-data-extraction-from-unstructured-pdfs.
18. Patil, M. 2025. Enhancing RAG Performance with Knowledge Graphs. *Medium*. Posted 25 April. https://medium.com/@manjunathpatil/enhancing-rag-performance-with-knowledge-graphs-abc123.
19. Microsoft Corporation. 2025. Retrieval Augmented Generation (RAG) in Azure AI Search. *Microsoft Learn Documentation*. Accessed 5 August. https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation.
20. Greyling, C. 2024. Corrective Retrieval Augmented Generation (CRAG). *Medium*. Posted 5 February. https://cobusgreyling.medium.com/corrective-retrieval-augmented-generation-crag-429.
21. Kirkovska, A. 2025. Document Data Extraction in 2025: LLMs vs OCRs. *Vellum Blog*. Posted 16 May. https://www.vellum.ai/blog/document-data-extraction-in-2025-llms-vs-ocrs.
22. BaseCap Analytics. 2025. The 3% OCR Accuracy Gap: Why Document Recognition Solutions Need an Intelligent Validation Layer. *BaseCap Blog*. Accessed 5 August. https://www.basecapanalytics.com/blog/the-3-percent-ocr-accuracy-gap.
23. Martinez, J. 2025. 67 Data Entry Statistics for 2025. *DocuClipper Blog*. Posted 5 March. https://www.docuclipper.com/blog/67-data-entry-statistics-for-2025.
24. Huang, Y., Lv, T., Cui, L., Lu, Y., & Wei, F. (2022). LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking. Proceedings of the 30th ACM International Conference on Multimedia (MM '22), 10–14 Oct, Lisboa, Portugal. https://doi.org/10.1145/3503161.3548112